

FOR APPLIED AI ENGINEERS

Fundamentals of Large Language Models



Deep Concepts, Zero Math — All Analogies

What We'll Cover Today

01

What is an LLM?

The Library Analogy

02

Tokens & Embeddings

LEGO Bricks & GPS Coords

03

Attention Mechanism

The Detective Analogy

04

Training (Pre-train & Fine-tune)

Scholar → Specialist

05

RLHF & Alignment

Human Feedback Loop

06

Context, Temperature & Sampling

Working Memory & Creativity

07

Hallucinations & RAG

Confident Guesser & Reference Books

08

Agents & Tool Use

The AI Co-worker



What Is a Large Language Model?

The Library Analogy

Imagine a scholar who has read **every book, article, and webpage ever written**.

They didn't memorise the words — they absorbed **patterns of language, knowledge, and reasoning**.

Ask them anything, and they compose a fluent answer — drawing on what they've seen, not looking it up word-for-word.

Parameters

Billions of learned numerical weights — the 'memories' of the model

LLM have a giant database (Vocab Dict)

LLMs look up Vocab dictionary — they generate them token by token

Probabilistic

Every output is a probability distribution over possible next words

Emergent skill

Capabilities like reasoning & coding emerge from scale alone

Tokens — The Atomic Unit of Text



Think of tokens as LEGO bricks.

You don't feed the model characters or whole words — you feed it fragments. A long word might be 3 tokens; a short common word is 1.

Un

believ

able

→ 3 tokens

12
34

~1 token ≈ 4 chars

GPT-4 uses ~100K vocab. Every text you send/receive is billed in tokens.



Language matters

English is more token-efficient than most other languages — same meaning, fewer tokens.



Tokens = money & speed

Fewer tokens in your prompt = cheaper & faster inference. Compress your prompts.



Context is token-limited

The model only sees N tokens of context. Old info gets silently dropped.

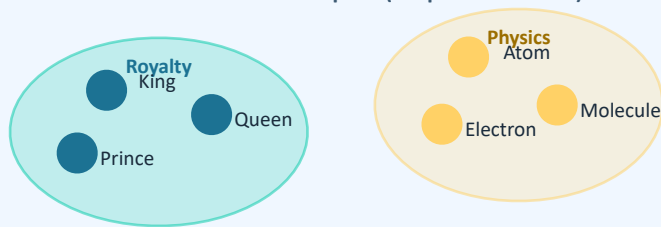
Embeddings — Meaning as Location in Space



Embeddings are GPS coordinates for words.

Each token is mapped to a point in a high-dimensional space. Words with similar meanings cluster together — just as nearby GPS coordinates describe nearby places.

Semantic Space (simplified 2D view)



Clusters = similar meaning

King – Man + Woman $\approx$ Queen

Arithmetic on embeddings works! Meaning is geometry. Relationships are encoded as directions in space.

Similarity = closeness

Two tokens close in space mean similar things. Cosine similarity measures this distance — it powers search, RAG, and recommendations.

Dimensions $\neq$ interpretable

There are 768–4096 dimensions. No single dim means 'royalty'. The meaning is spread across ALL dimensions together.

Embeddings are learnt

The model learns these coordinates during training. They're not hand-crafted — they emerge from reading trillions of words.

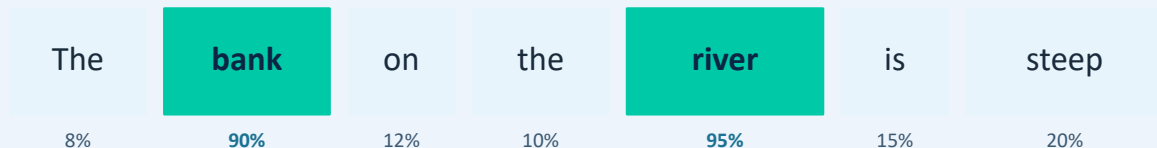
Attention — How the Model Reads Context



Attention works like a detective reading a witness statement.

When parsing each word, the detective highlights every other word it needs to understand the current one — and ignores the rest. That selective focus is Attention.

Attending to "bank" — which other words get focus?



↑ *attention score*



Query, Key, Value

Q = what I'm looking for, K = what each token offers, V = what I actually use. Like a search engine for each word.



Multi-Head Attention

Run 8–96 attention operations in parallel. Each 'head' looks at different relationships simultaneously.

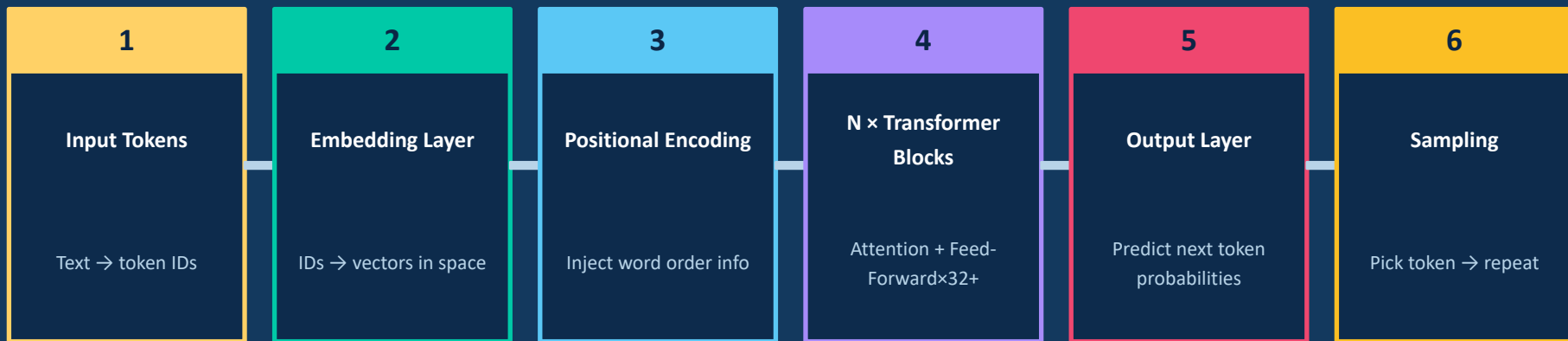


Scales with context

Attention looks at ALL tokens in context. Longer context = more computation — quadratic cost is the bottleneck.

Transformer Architecture

The assembly line that converts tokens to answers



Key Insight:

Transformer Blocks stack 32–96 times. Each layer refines understanding — early layers detect grammar, later layers reason about meaning and world knowledge.

Pre-Training — Reading the Whole Internet

 **Pre-training is like a person reading every book in history — cover to cover — for years.**

The goal: predict the next word in any sentence. That's it. From this deceptively simple task, all intelligence emerges.



Scale

GPT-4 trained on ~13 trillion tokens.
A human reads ~100M words in a lifetime.
The model reads 100,000× more.



Objective

Predict the next token.
This forces the model to learn grammar, facts, logic,
and reasoning — as side effects.



Cost

Training GPT-4 cost est. \$100M.
Inference (running it) is cheap by comparison.
Pre-trained weights are reused for everything.



What's Learnt

Syntax, semantics, world knowledge, common sense,
code, math reasoning — all from one task.



Duration

Weeks to months on thousands of GPUs.
Once done, the base model is frozen —
you don't re-train from scratch.



Limitation

Pre-trained models are 'raw scholars' — brilliant but
unfiltered. They'll say anything plausible, true or not.

Fine-Tuning — From Scholar to Specialist



Fine-tuning is like sending a brilliant generalist to a professional residency.

The core knowledge stays. You just teach them the norms, style, and domain of a specific job.

Instruction Fine-Tuning (SFT)

- Show the model thousands of (prompt, ideal-response) pairs
- Model learns to follow instructions — not just predict next tokens
- This is how a base model becomes a 'chat' model
- Examples: 'Summarise this text' → [good summary]

Domain Fine-Tuning

- Continue training on domain-specific data (medical, legal, code)
- Model gets much better at that domain's language & concepts
- Risk: catastrophic forgetting of general knowledge
- Use LoRA/QLoRA for efficient fine-tuning on consumer hardware

Parameter-Efficient FT (LoRA)

- Freeze 99% of the model's weights
- Only train small 'adapter' layers added on top
- Analogy: fitting a specialist lens on a camera — don't rebuild the camera
- 10–100× cheaper than full fine-tuning

RLHF — Teaching the Model What 'Good' Means



RLHF is like training a dog — except the dog already knows everything, and you're just teaching it manners.

Reinforcement Learning from Human Feedback aligns model outputs with human preferences: helpful, harmless, and honest.

1



**Generate
Responses**

Model produces many answers to the same prompt



2



**Human
Ranking**

Humans rank responses: $A > B > C$



3



**Train Reward
Model**

A separate model learns what 'good' means from rankings



4



**RL Optimisation
(PPO)**

Main model is updated to score higher on reward model



Why it matters for engineers: RLHF is why the same base model can be 'GPT-4' (polite, safe) or a raw base model (unpredictable). System prompts + RLHF shape 90% of the UX.

Context Window & Temperature



Context Window

Think of it as Working Memory

A human can hold ~7 items in working memory. An LLM can hold 8K–1M tokens — but only what fits. Anything older is gone.

Your context window is like RAM:



← 100% of context window →

- Longer ≠ always better — very long contexts dilute attention
- Recent tokens get more 'attention weight' naturally
- Sliding window / summarisation = manage long conversations



Temperature

The Creativity Dial

Temperature controls randomness in token selection. Low = safe & predictable. High = creative & surprising (sometimes wrong).

T = 0.0

Always picks the highest-probability token. Fully deterministic. Great for code, SQL, structured output.

T = 0.7

Balanced. Good for chat, summaries, and most use-cases. Some variety, still coherent.

T = 1.5

Wild creativity. May hallucinate or go off-topic. Good for brainstorming, creative writing.

Hallucinations — Why Models Confidently Lie



An LLM is a statistical next-token predictor — not a truth engine.

Hallucination = when a statistically plausible sequence is factually wrong. The model doesn't 'know' it's lying — it just keeps picking likely next tokens.



Factual Errors

"The Eiffel Tower was built in 1832."

Why:

Training data conflict; the model interpolates plausibly between sources

Fix:

Ground with RAG / verified sources for factual tasks



Confabulated Citations

"See Smith et al. (2021)..." — paper doesn't exist

Why:

Generating a plausible-looking citation pattern it's seen millions of times

Fix:

Never trust model-generated references without verification



Confident Extrapolation

Answering a question beyond its knowledge cutoff with false certainty

Why:

No internal uncertainty signal — it generates tokens, not beliefs

Fix:

Include knowledge cutoff info; use date-aware prompting

RAG — Give the Model a Reference Library



RAG = Retrieval Augmented Generation.

Instead of relying on the model's (possibly stale or wrong) memory — retrieve the right documents first, then ask the model to reason over them.



User Query

"What's our refund policy?"



Embed & Retrieve

Query → vector → search doc
store



Top-K Chunks

3-5 relevant policy paragraphs
returned



Augment Prompt

Chunks injected into context as
'reference'



Grounded Answer

Model reasons over real policy
text



RAG Strengths

- Reduces hallucinations on factual Q&A
- Answers grounded in real documents
- Knowledge cutoff no longer a blocker



RAG Pitfalls

Retrieval quality is the bottleneck · Bad chunks confuse the model · Tune chunk size carefully

Agents & Tool Use

The LLM as an AI Co-worker, not just a responder

🤝 An agent is an LLM in a loop — it can reason, choose tools, execute, observe results, and keep going until the task is done.



Think

Reason about the goal and what action to take next



Act

Call a tool: web search, code execution, API, DB query



Observe

Read the tool output back into the context window



Repeat

Continue until the task is complete or a stop condition is met



Tool Calling

Model emits structured JSON to call functions. The runtime executes the real action (not the model).



Planning

Chain-of-thought prompting + structured plans. ReAct, Tree-of-Thought, and Plan-and-Execute patterns.

What Every AI Engineer Should Remember



1

LLMs are next-token predictors — everything else is an emergent property of scale.

2

Tokens are the atomic unit — optimise your prompts to minimise token waste.

3

Attention is why context matters — but long contexts have quadratic cost.

4

Fine-tune to specialise; LoRA to do it cheaply without catastrophic forgetting.

5

RLHF shapes behaviour — your system prompt fights/works with alignment.

6

Hallucinations are structural, not bugs — mitigate with RAG + verification.

7

Agents let models act — but plan for failure; every tool call can go wrong.